

LAB 4

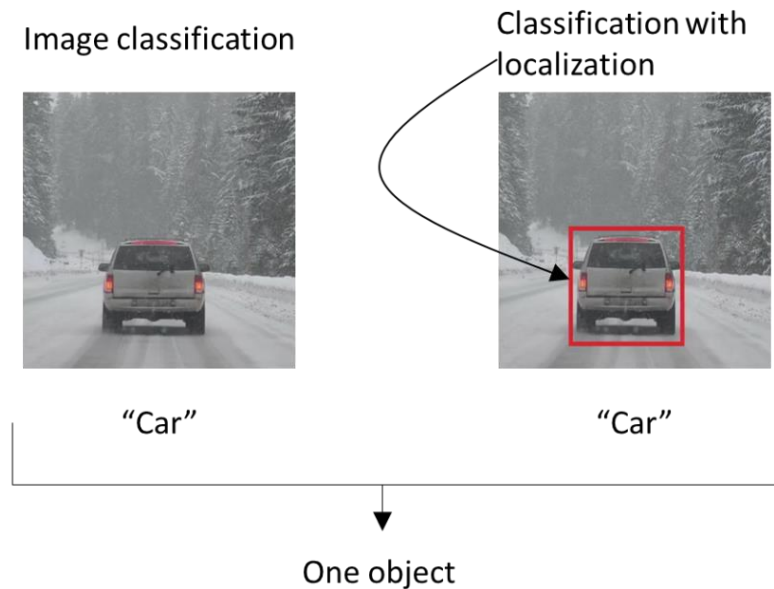
I. Object Localization

Bài toán

Object Localization yêu cầu mô hình xác định **vị trí** và **nhãn** của một object trong ảnh.

Input là một ảnh RGB.

Output gồm hai phần: **nhãn lớp** và **một bounding box** biểu diễn dưới dạng (cx, cy, w, h) - tọa độ tâm, chiều rộng, chiều cao.

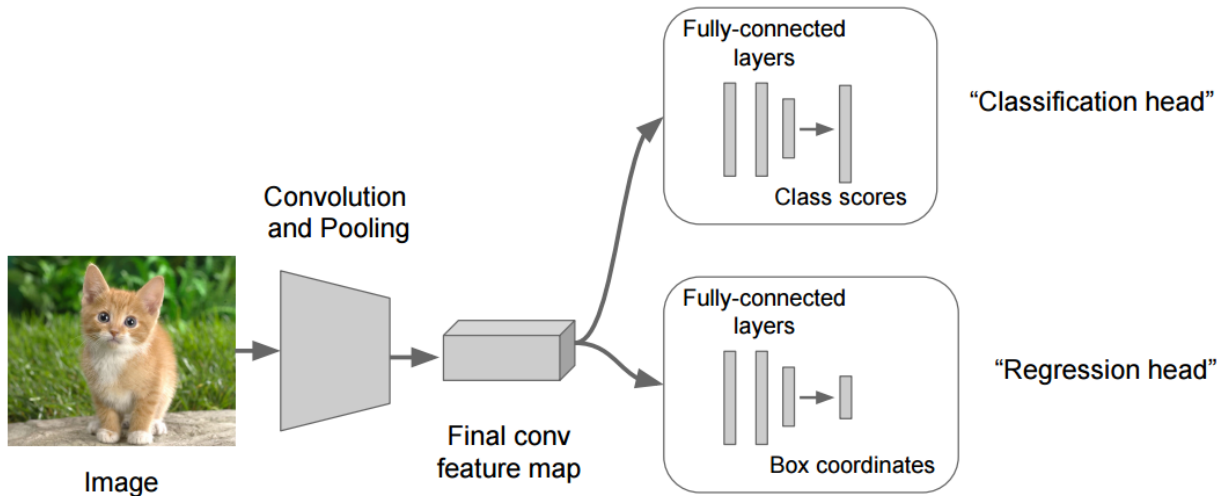


Ý tưởng

Localization cần giải quyết hai bài toán con cùng lúc: **phân loại** (object này là gì?) và **hồi quy tọa độ** (object này ở đâu?). Hai bài toán này không độc lập các feature giúp nhận biết "đây là đối tượng" cũng chứa thông tin về hình dạng và vị trí của nó. Do đó, thay vì huấn luyện hai mô hình riêng, ta dùng một backbone chung để trích xuất feature, sau đó tách ra hai nhánh xử lý độc lập.

Đây là **multi-task learning** - một backbone chia sẻ representation cho cả hai tác vụ. Cách này hiệu quả hơn về mặt tính toán và thường cho kết quả tốt hơn vì gradient từ cả hai task cùng cập nhật backbone.

Sau khi backbone trích xuất feature vector, mô hình tách thành hai nhánh.



Classification head học cách trả lời "*object này là gì*". Nó nhận feature vector, đưa qua các lớp linear, và output xác suất thuộc 21 lớp. Loss là Cross-Entropy giúp tăng xác suất đúng lớp lên cao, các lớp còn lại xuống thấp.

Regression head học cách trả lời "*object này ở đâu*". Cùng feature vector đó, nhánh này output 4 số (cx, cy, w, h). Loss là Smooth-L1 để đo khoảng cách giữa box dự đoán và ground-truth.

Vì cả hai nhánh dùng chung feature, gradient từ cả hai loss cùng cập nhật ngược về backbone. Backbone không chỉ học để phân loại tốt, cũng không chỉ học để định vị tốt và bị ràng buộc phải học feature hữu ích cho cả hai cùng lúc. Đó là lợi ích cốt lõi của multi-task learning trong trường hợp này.

Code chính triển khai cho ý tưởng trên:

```
class LocalizationNet(nn.Module):
    def __init__(self, num_classes=NUM_CLASSES, pretrained=True):
        super().__init__()
        backbone = models.resnet50(weights="IMAGENET1K_V1" if pretrained else None)
        self.features = nn.Sequential(*list(backbone.children())[:-1]) # → (B, 2048, 1, 1)
        feat_dim = 2048

        self.cls_head = nn.Sequential(
            nn.Flatten(),
            nn.Linear(feat_dim, 256), nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, num_classes)
        )
        self.reg_head = nn.Sequential(
            nn.Flatten(),
            nn.Linear(feat_dim, 256), nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, 4),
            nn.Sigmoid() # output in (0,1)
        )

    def forward(self, x):
        f = self.features(x)
        return self.cls_head(f), self.reg_head(f)
```

Kết quả minh họa:

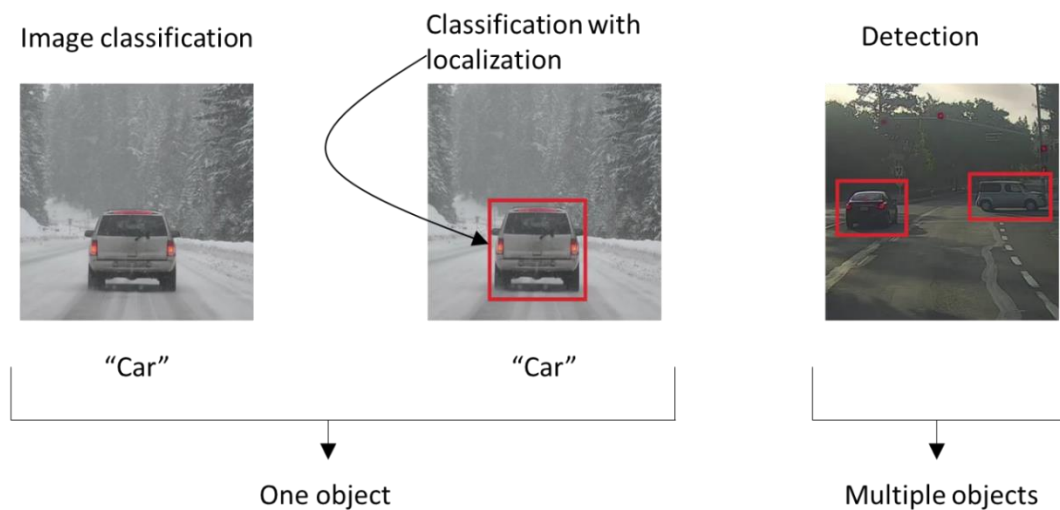


II. Bài toán Object Detection

Faster RCNN

Bài toán

Faster R-CNN giải quyết object detection, bài toán này khác với localization ở chỗ một ảnh có thể chứa nhiều object, mỗi object cần một box và một nhãn riêng. Số lượng object trong mỗi ảnh không cố định, nên output không thể là một vector kích thước cố định như localization.

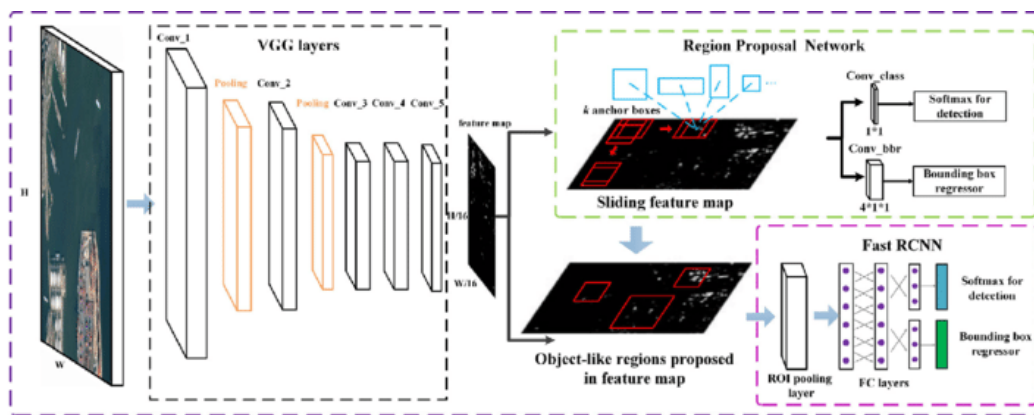


Vấn đề gặp phải

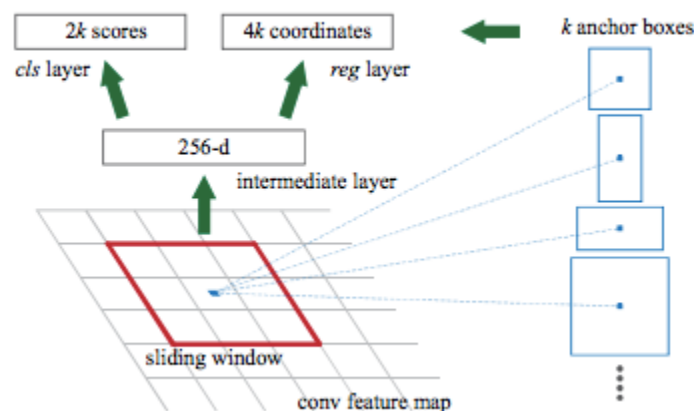
Trước Faster R-CNN, pipeline detection gồm hai bước tách rời: đầu tiên dùng thuật toán truyền thống như Selective Search để tạo ra ~2000 vùng ảnh có thể chứa object (region proposals), sau đó đưa từng vùng qua CNN để phân loại. Vấn đề là Selective Search chạy trên CPU, mất ~2 giây mỗi ảnh, trở thành bottleneck của toàn bộ pipeline). Hai bước này hoàn toàn độc lập, CNN không có cách nào ảnh hưởng đến chất lượng của proposals.

Cách Faster R-CNN xử lý

Ý tưởng cốt lõi là thay Selective Search bằng một mạng nhỏ chạy trên chính feature map của backbone gọi là **Region Proposal Network (RPN)**. Vì RPN dùng chung feature với detection head, toàn bộ pipeline có thể train end-to-end.



RPN trượt một cửa sổ nhỏ trên feature map, tại mỗi vị trí dự đoán k anchor boxes với các tỉ lệ và kích thước khác nhau. Với mỗi anchor, RPN trả lời hai câu hỏi: đây có phải vùng chứa object không (objectness score), và cần điều chỉnh box bao nhiêu để khớp hơn với object thật (box regression).

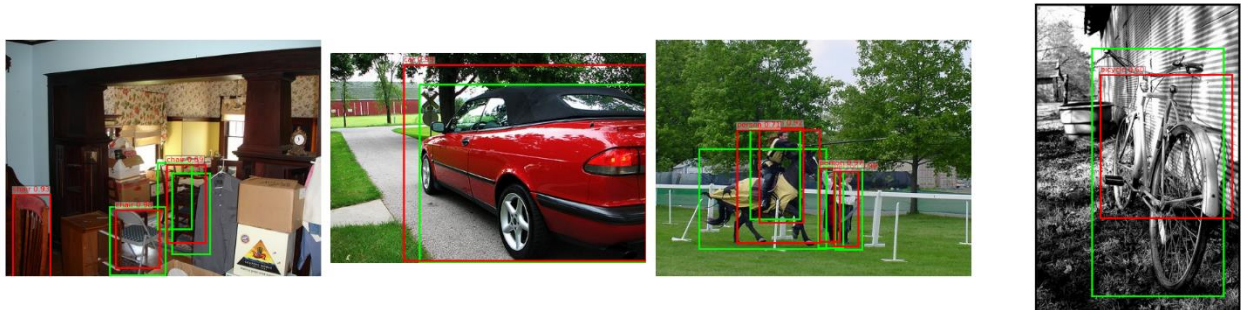


Các proposals từ RPN sau đó được chuẩn hóa về cùng kích thước bằng **RoI Pooling**, rồi đưa qua detection head cũng gồm classification head và regression head như localization, nhưng chạy độc lập cho từng proposal.

Code triển khai:

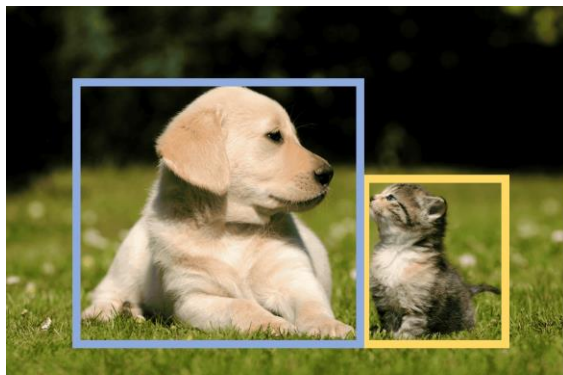
```
def get_model():
    model = fasterrcnn_resnet50_fpn(weights="DEFAULT")
    in_f = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_f, NUM_CLASSES)
    return model
```

Kết quả minh họa:



III. Segmentation

Bài toán segmentation yêu cầu gán nhãn cho từng pixel trong ảnh, output là một mask có cùng kích thước $H \times W$ với input.



Object Detection

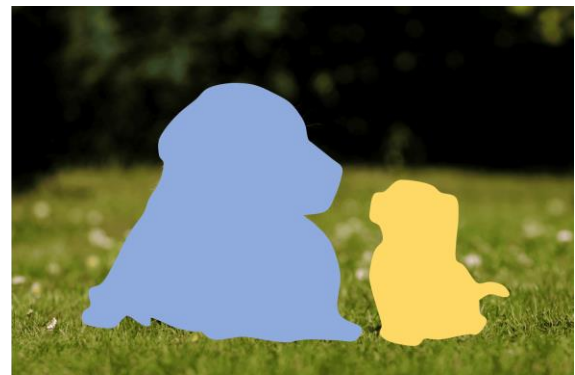
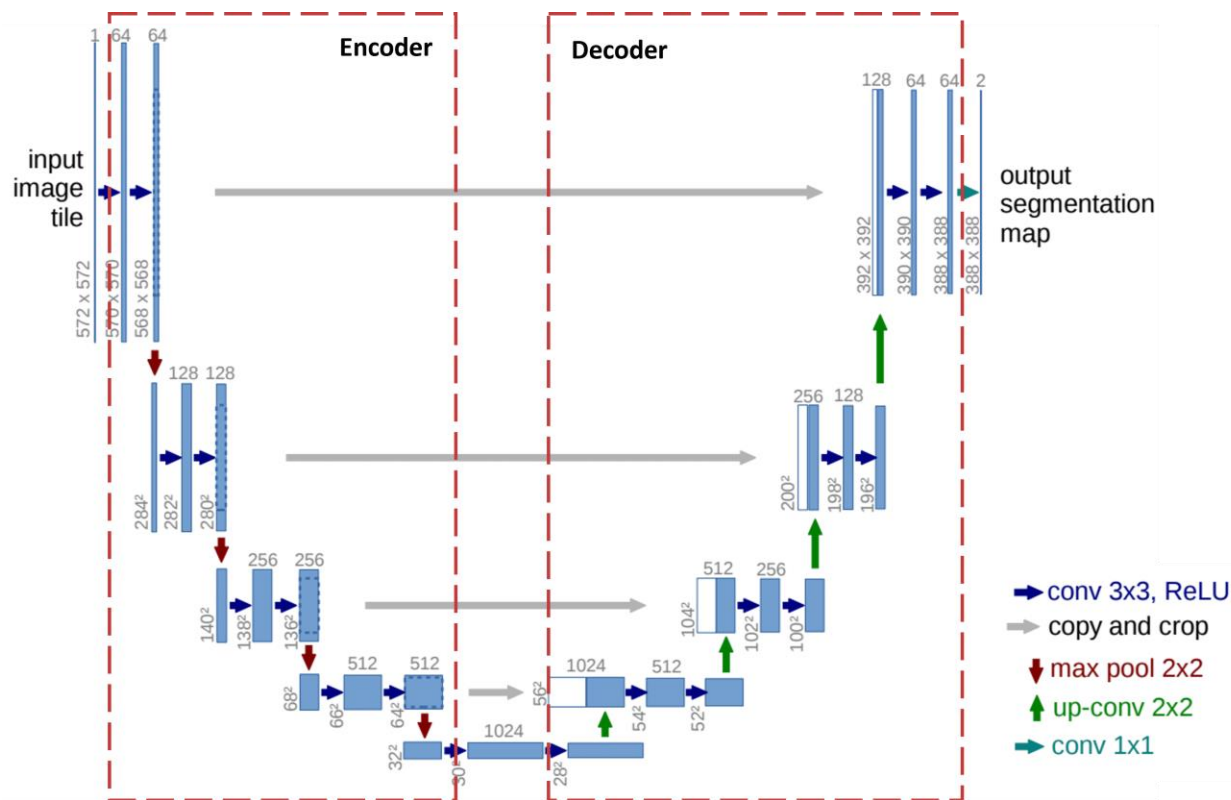


Image Segmentation

Ràng buộc này khiến các CNN encoder thông thường không dùng trực tiếp được vì chúng nén spatial size xuống quá nhỏ mà không có cơ chế khôi phục lại.

Unet

Encoder nhận ảnh đầu vào và qua từng block Conv và MaxPool, giảm dần H, W trong khi tăng số channel. Kết quả là tầng nông giữ thông tin low-level (cạnh, texture), tầng sâu giữ thông tin high-level (hình dạng, loại object), nhưng thông tin vị trí chính xác từng pixel bị mất dần.



Bottleneck là điểm cuối của encoder feature size nhỏ nhất, channel depth lớn nhất. Đây là nơi model nắm context toàn cục của ảnh trước khi bắt đầu tái tạo mask.

Decoder upsample từng bước để khôi phục spatial resolution về kích thước gốc. Upsampling đơn thuần không đủ vì thông tin ranh giới đã mất qua pooling. Skip connection giải quyết điều này bằng cách concat feature map từ encoder (trước bước pool, còn giữ spatial detail) vào decoder ở cùng resolution.

Output cuối là một Conv 1x1 chiếu về số class, qua sigmoid hoặc softmax ra xác suất, rồi threshold để ra mask nhị phân. Mỗi pixel trong mask là kết quả tổng hợp của toàn bộ pipeline: encoder hiểu nội dung, bottleneck nắm context, decoder và skip tái tạo ranh giới chính xác ở mức pixel.

Khối tích chập sử dụng:

```
class DoubleConv(nn.Module):
    """(Conv2d → BN → ReLU) × 2"""
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, 3, padding=1, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, 3, padding=1, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
        )
    def forward(self, x): return self.block(x)
```

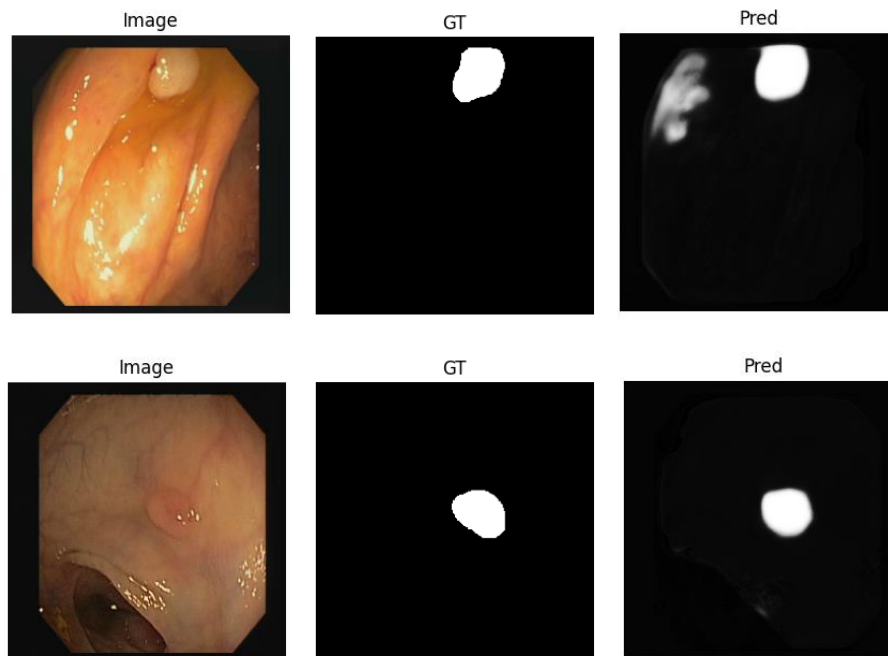
Kiến trúc Unet triển khai:

```
class UNet(nn.Module):
    def __init__(self, in_ch=3, out_ch=1, features=(64,128,256,512)):
        super().__init__()
        # Encoder
        self.enc = nn.ModuleList()
        self.pool = nn.MaxPool2d(2, 2)
        ch = in_ch
        for f in features:
            self.enc.append(DoubleConv(ch, f))
            ch = f
        # Bottleneck
        self.bottleneck = DoubleConv(ch, ch*2)
        ch = ch*2
        # Decoder
        self.dec_up = nn.ModuleList()
        self.dec_conv = nn.ModuleList()
        for f in reversed(features):
            self.dec_up.append(
                nn.ConvTranspose2d(ch, f, kernel_size=2, stride=2)
            )
            self.dec_conv.append(DoubleConv(f*2, f)) # f*2 because of skip concat
            ch = f
        # Head
        self.head = nn.Conv2d(ch, out_ch, 1)

    def forward(self, x):
        skips = []
        for enc in self.enc:
            x = enc(x)
            skips.append(x)
            x = self.pool(x)
        x = self.bottleneck(x)
        skips = skips[::-1]
        for i, (up, conv) in enumerate(zip(self.dec_up, self.dec_conv)):
            x = up(x)
            s = skips[i]
            if x.shape != s.shape: # pad if needed
                x = TF.pad(x, [0, s.shape[3]-x.shape[3],
                               0, s.shape[2]-x.shape[2]])
            x = torch.cat([s, x], dim=1)
            x = conv(x)
        return self.head(x)
```

Link dataset: <https://www.kaggle.com/datasets/balraj98/cvcclinicdb>

Kết quả minh hoạ:



III. Contrastive Language-Image Pre-Training

CLIP là một mô hình học đa phương thức được phát triển bởi OpenAI, với mục tiêu xây dựng một không gian biểu diễn chung cho cả dữ liệu hình ảnh và văn bản. Thay vì chỉ xử lý một loại dữ liệu như các mô hình truyền thống, CLIP có khả năng hiểu và liên kết thông tin giữa hai miền khác nhau, từ đó mở ra khả năng giải quyết nhiều bài toán kết hợp giữa thị giác máy tính và xử lý ngôn ngữ tự nhiên.

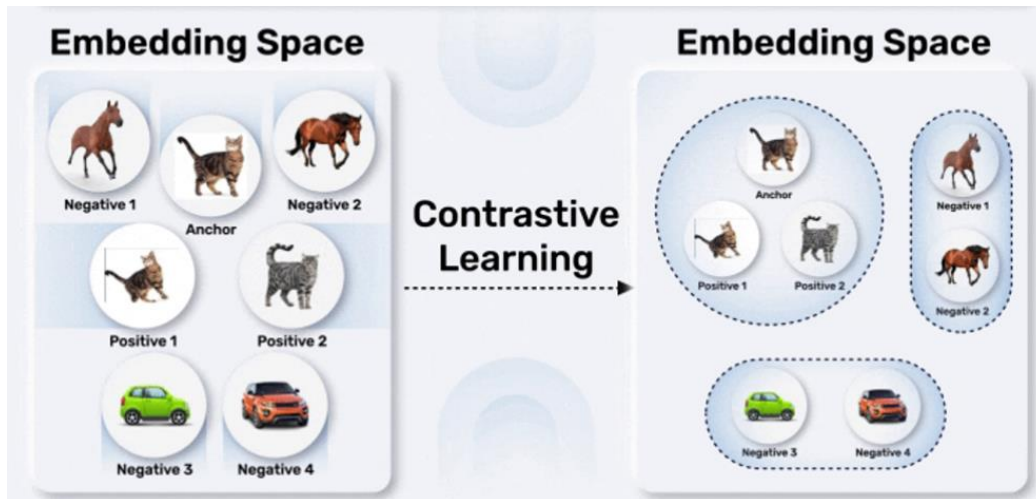
Ý tưởng

Ý tưởng chính của CLIP là tận dụng lượng dữ liệu khổng lồ trên internet dưới dạng các cặp ảnh và mô tả tương ứng. Thay vì sử dụng nhãn thủ công, mô hình học trực tiếp từ các mô tả tự nhiên này để hiểu nội dung hình ảnh. Bằng cách học cách liên kết ảnh với các câu mô tả đúng và phân biệt với các mô tả sai, CLIP xây dựng được khả năng hiểu ngữ nghĩa ở mức cao, gần với cách con người mô tả thế giới.

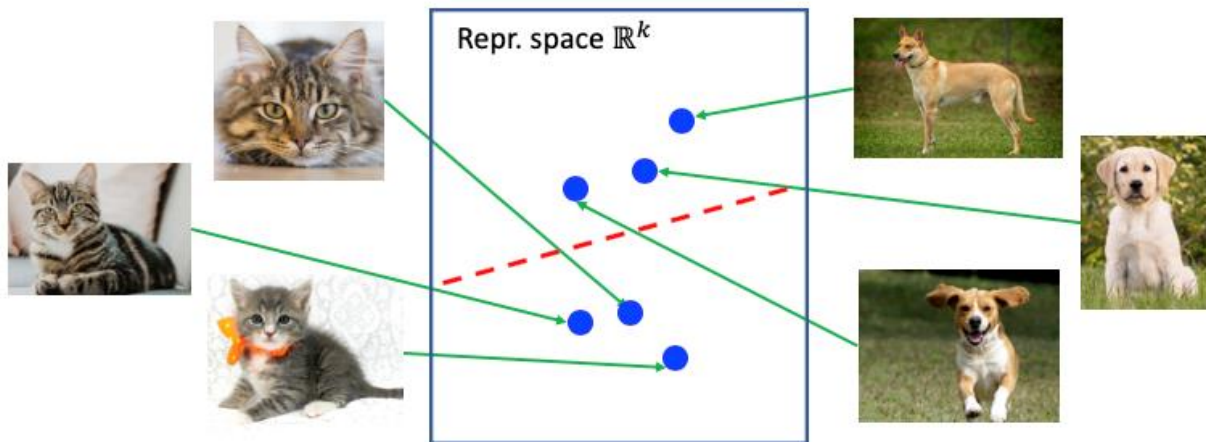
Cơ chế hoạt động

CLIP hoạt động dựa trên phương pháp học tương phản (contrastive learning).

Contrastive Learning là một phương pháp học biểu diễn trong đó mô hình học cách phân biệt giữa các cặp dữ liệu giống nhau (positive pairs) và khác nhau (negative pairs).



Thay vì học trực tiếp từ nhãn lớp như trong supervised learning, phương pháp này tập trung vào việc đưa các biểu diễn của dữ liệu “liên quan” lại gần nhau trong không gian vector, đồng thời đẩy các biểu diễn “không liên quan” ra xa nhau.



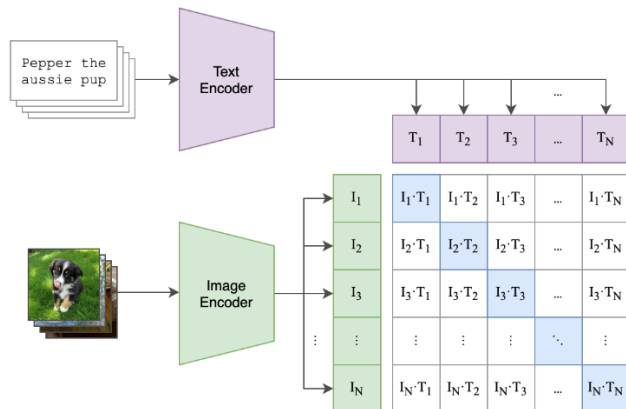
Kiến trúc mô hình

Trong quá trình huấn luyện, mỗi ảnh sẽ được ghép với một mô tả đúng, đồng thời so sánh với nhiều mô tả sai khác trong cùng một batch. Mô hình sẽ tối đa hóa độ tương đồng giữa cặp đúng và giảm độ tương đồng giữa các cặp sai. Nhờ đó, CLIP học được cách ánh xạ ảnh và văn bản vào cùng một không gian vector, nơi các cặp liên quan sẽ nằm gần nhau hơn.

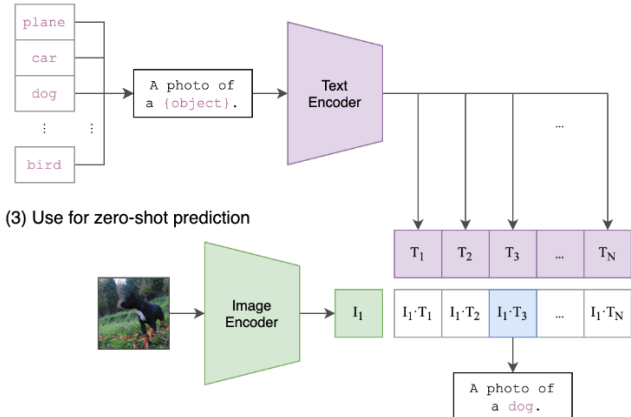
CLIP bao gồm hai thành phần chính: một bộ mã hóa ảnh (image encoder) và một bộ mã hóa văn bản (text encoder). Bộ mã hóa ảnh có thể sử dụng các kiến trúc như ResNet hoặc Vision Transformer để trích xuất đặc trưng từ ảnh, trong khi bộ mã hóa văn bản sử dụng

Transformer để xử lý chuỗi từ. Hai đầu ra này được đưa về cùng một không gian embedding và được chuẩn hóa trước khi tính toán độ tương đồng bằng cosine similarity.

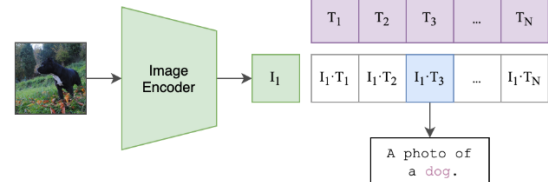
(1) Contrastive pre-training



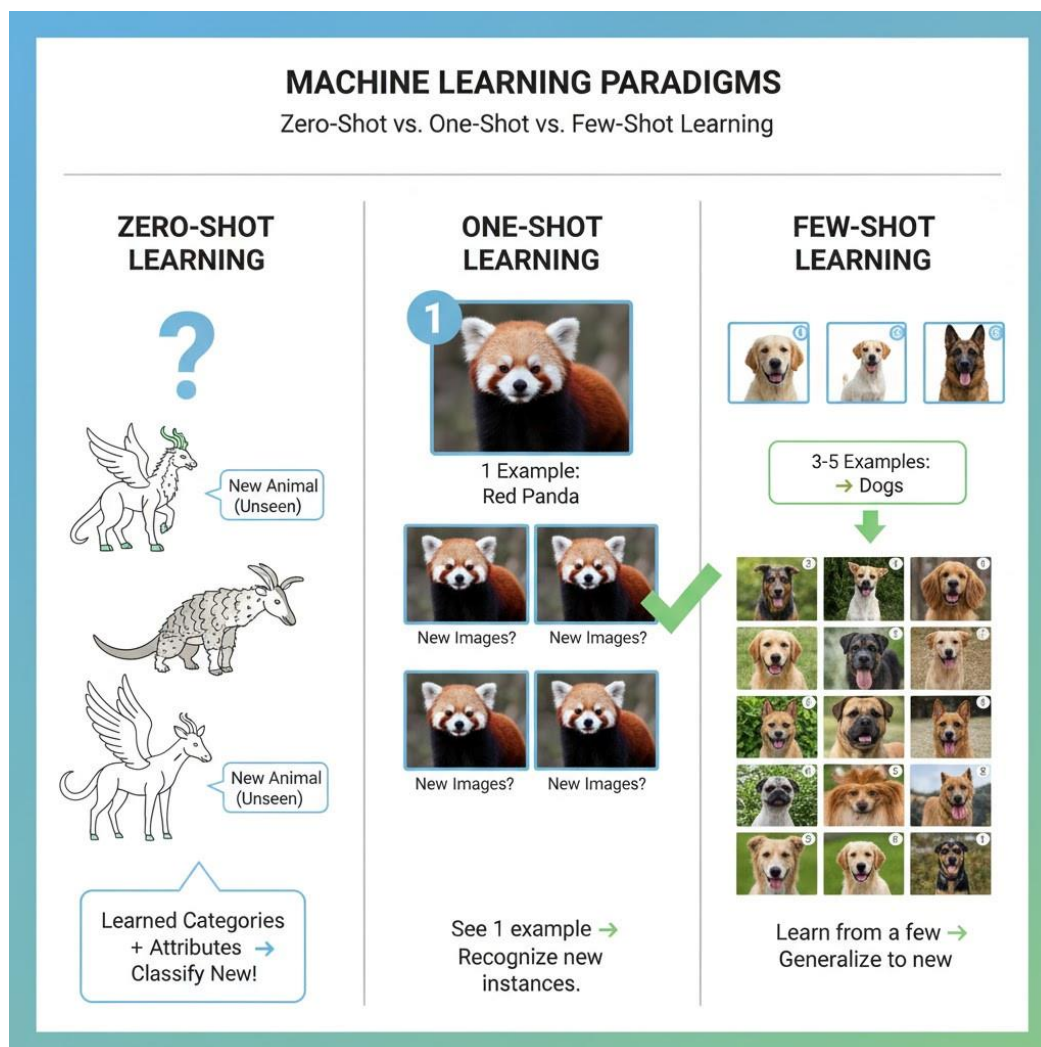
(2) Create dataset classifier from label text



(3) Use for zero-shot prediction



Low-shot learning:



1. Zero-shot

Zero-shot là khi mô hình **không được huấn luyện trước trên lớp dữ liệu đó** nhưng vẫn có thể dự đoán.

Ví dụ với CLIP:

- Không train trên lớp “zebra”
- Nhưng vẫn nhận diện được nhờ mô tả: “*a photo of a zebra*”

Không cần sample cho mỗi lớp.

2. One-shot

One-shot là khi mô hình chỉ cần **1 ví dụ duy nhất** để học một lớp mới.

Ví dụ:

- Chỉ cần 1 ảnh của một người → nhận diện lại người đó

Cần 1 sample cho mỗi lớp.

3. Few-shot

Few-shot là khi mô hình học với **một số ít dữ liệu** (thường 2–10 mẫu mỗi lớp).

Ví dụ:

- 5 ảnh con mèo → học phân biệt mèo với các lớp khác

Cần một vài sample (k nhỏ)